MGL 7230: Tests Logiciels

Lecture # 6

Tests de Régression

Based on Chapter 9 from Mathur, Aditya P. Foundations of software testing, 2/e. Pearson, 2013.

Plan

- Qu'est-ce qu'un test de régression ?
- Processus de test de régression
- Gestion des tests dans une suite de tests de régression
- Problème de sélection de tests de régression (RTS)
- Revalidation des tests
- Sélection des tests de traversée de modification
- Minimisation des tests
- Priorisation des tests
- Support d'outils automatisés

Tests de Régression - Définition

- La plupart des logiciels actuels ont très peu de nouveaux développements
  - Corriger, perfectionner, adapter ou prévenir les problèmes des logiciels existants
  - Créer de nouveaux programmes à partir de composants existants
  - Adapter des logiciels existants à de nouvelles situations

Définition

Processus de retest d'un logiciel modifié

Tests de régression

Processus de développement, de test et de déploiement simplifié

Version 1	Version 2
1. Develop *P*	4. Modify *P* to *P'*
2. Test *P*	5. Test *P'* for new functionality
3. Release *P*	6. Perform regression testing on *P'* to ensure that the code carried over from *P* behaves correctly
	7. Release *P'*

Les développeurs peuvent **ajouter** ou **supprimer** par erreur du code qui **empêche la fonctionnalité existante et inchangée de P** de se comporter comme souhaité.

Tests de régression

- Les tests de régression peuvent **intervenir à n'importe quelle phase du développement logiciel**.
  - Ils ne constituent **pas nécessairement une phase de test distincte**.

- Ils peuvent être nécessaires en cas de **modification du hardware** correspondant.
  - Dans ce cas, des tests de régression sont effectués même s'il n'y a pas de modifications dans le logiciel.

Tests de régression

- En raison des **interconnexions profondes** entre les composants logiciels,
  - les modifications apportées à une méthode peuvent entraîner des problèmes dans des méthodes apparemment sans rapport.
- Sans surprise, la majeure partie de nos tests sont des tests de régression.

- De grandes suites de tests de régression s'accumulent à mesure que les programmes (et les composants logiciels) vieillissent.

Une approche simple

Peut-on simplement réexécuter tous les tests développés pour la version précédente ?

Une approche simple

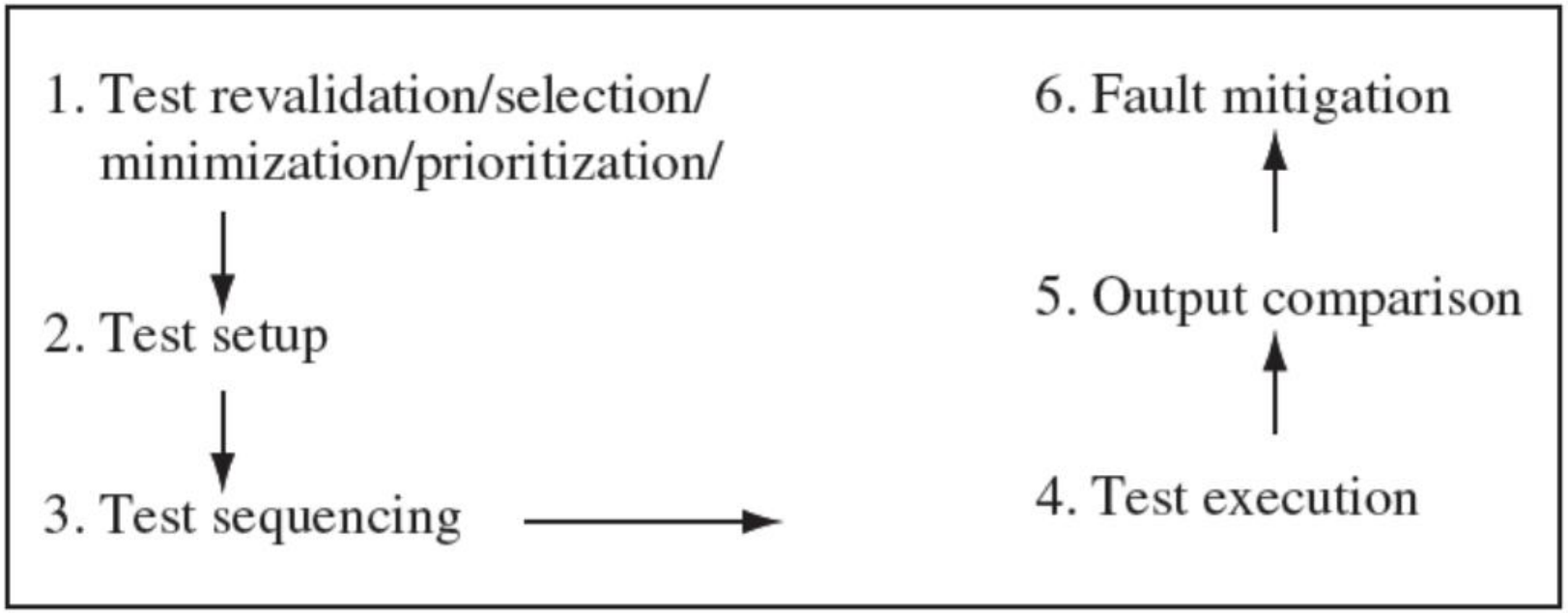
Problèmes potentiels :

- Il se peut que vous n'ayez pas **le temps** d'effectuer tous les tests.
- Certains tests peuvent **ne plus être valides**, par exemple si vous modifiez le **nombre de paramètres d'entrée** ou **le format des données d'entrée**.
- Cela peut ne pas être efficace, car **certains tests peuvent ne pas avoir de lien avec ces changements**.

Tâches majeures

- La **revalidation** de test fait référence à la tâche de vérifier quels tests pour P restent valides pour P'.
- La **sélection** des tests fait référence à l'identification des tests qui parcourent les parties modifiées de P'.
- La **minimisation** des tests fait référence à la suppression de tests apparemment redondants par rapport à certains critères.
- La **priorisation** des tests fait référence à la tâche de prioriser les tests selon certains critères.

Processus de test de régression



Regression Test Selection (RTS) Problem

- Le problème de sélection des tests de régression (Regression Test Selection (RTS)) consiste à identifier les tests à exécuter afin de tester la régression d'un programme modifié.

Revalidation des tests de régression

- La revalidation des tests consiste à vérifier quels tests pour P restent valides pour P'.

- La revalidation des tests est nécessaire pour garantir que **seuls les tests applicables à P'** sont utilisés lors des tests de régression.

- Les **tests validés** peuvent être redondants s'ils ne couvrent aucune des parties modifiées de P'.

Revalidation: exemple (1)

- Considérons un code postal de service web **ZipCode** qui offre deux services :
  - **ZtoC** : renvoie une liste des villes (Cities) et de l'État pour un code postal donné
  - **ZtoA** : renvoie l'indicatif régional (Area code) d'un code postal donné

- Supposons que Zip Code ne dessert initialement que les États-Unis, puis qu'il soit modifié comme suit :
  - **ZtoC** est modifié pour qu'un utilisateur doive fournir **un pays donné** ainsi qu'un **code postal**.
  - **ZtoT**, un nouveau service, est ajouté qui **saisit un pays** et un **code postal** et retourne **le fuseau horaire (Time)**.

- Notez que le code postal n'est pas universel (code postal à 6 caractères au Canada, contre 5 chiffres en USA).

Revalidation: exemple (2)

- Considérez les **deux tests suivants** (seules les entrées sont spécifiées) utilisés pour tester ZC :
  - t1 : <service=ZtoC, zip=47906>
  - t2 : <service=ZtoA, zip=47906>

- **Les deux tests ci-dessus peuvent-ils être appliqués à la nouvelle version ?**

Revalidation: exemple (3)

- t1 : <service=ZtoC, zip=47906>
- t2 : <service=ZtoA, zip=47906>

- Le test t1 n'est **pas valide** pour **ZtoC**, car il ne contient pas le **champ pays obligatoire**.
- Le test t2 est **valide**, car nous n'avons apporté aucune modification à **ZtoA**.
- Nous devons soit **abandonner t1** et **le remplacer par un nouveau test** pour le **ZtoC** modifié, soit simplement modifier t1 de manière appropriée.
  - t1 : <pays=États-Unis, service=ZtoC, code postal=47906>

Problème de sélection de test de régression

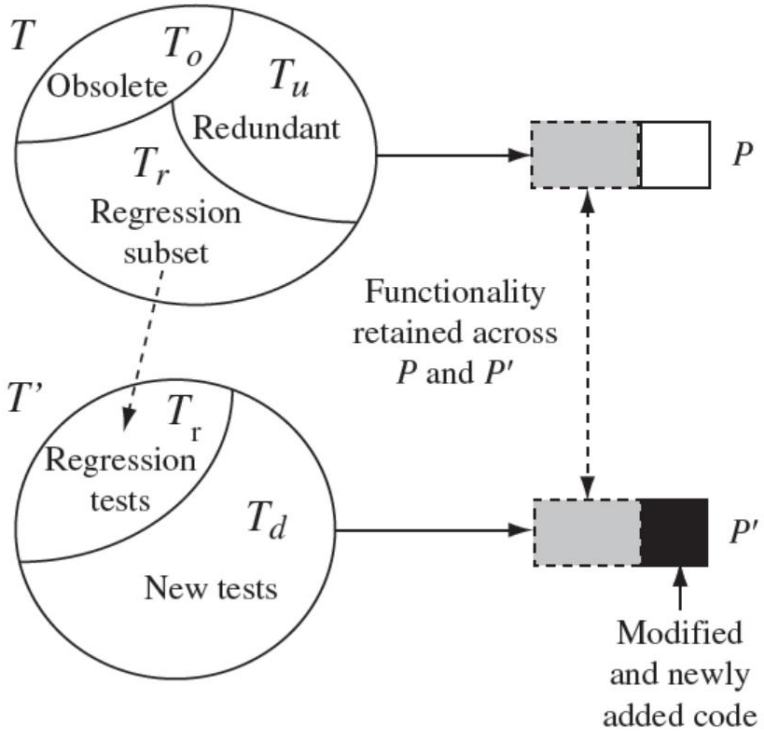


Figure 9.4 Tests de régression comme problème de sélection de tests. Un sous-ensemble Tr de l'ensemble T est sélectionné pour retester la fonctionnalité de P qui reste inchangée dans P'.

- Déterminer **Tr** de telle sorte que l'exécution réussie de **P'** par rapport à **Tr** implique que le code modifié ou nouvellement ajouté dans P' **n'a pas corrompu le code transféré de P**.
- Certains tests peuvent devenir **obsolètes** lorsque P est modifié en P'. **Ces tests ne sont pas inclus dans le sous-ensemble de régression Tr**.
- Cependant, une partie de **To** peut être modifiée pour devenir partie de **Tr** ou **Tu**.

Problème de sélection de test de régression

- Le problème RTS consiste à trouver un sous-ensemble minimal **Tr** de tests **non obsolètes** à partir de **T** tel que si **P'** réussit les tests dans **Tr**, alors il réussira aussi les tests dans **Tu**.
- Le problème de sélection du test de régression (RTS) est énoncé comme suit :

Trouver un Tr minimal, tel que:

$$\forall\, t \in Tr \text{ et } \forall\, t' \in Tu \cup Tr, \quad P(t) = P'(t) \Rightarrow P(t') = P'(t')$$

- Tester Tr contre P' équivaut à tester T contre P.

Remarque : la détermination de **Tr** nécessite que nous connaissions l'ensemble des tests obsolètes **To**.

Gestion des tests dans une suite de régression

- Les suites de tests accumulent de nouveaux tests au fil du temps.
- Les suites de tests sont généralement exécutées sur une période courte et fixe.
  - Souvent pendant la nuit, parfois plus fréquemment, parfois moins fréquemment.
- À un moment donné, le nombre de tests peut devenir ingérable.
  - Nous ne pouvons pas terminer l'exécution des tests dans le temps imparti.

Gestion des tests dans une suite de régression

- On peut toujours ajouter du matériel informatique.
- Mais est-ce que ça vaut le coup ?
- Combien de tests faut-il vraiment exécuter ?
- Plusieurs approches existent :
  - Tester tout
  - Sélection aléatoire de tests
  - Sélection de tests traversant les modifications
  - Minimisation des tests
  - Priorisation des tests
  - Découpage de traces et découpage dynamique (non abordés dans ce cours)

Approche: Tester Tout

- Tous les tests **valides** de la **version précédente** et les nouveaux tests créés pour tester toute fonctionnalité ajoutée.
- T' =T −To
- **L'approche la moins risquée**
- Mais que faire si les **ressources sont limitées pour exécuter des tests T-To** et que vous **devez respecter un délai ?**
  - Nous aurons peut-être besoin d'une suite de tests de régression **plus petite**
- Néanmoins, la stratégie de « tester tout » combinée à des outils de test de régression automatisés est peut-être la technique la plus utilisée dans le monde commercial.

Sélection aléatoire des tests

- Les tests sont sélectionnés aléatoirement dans l'ensemble T-T0.
- Le testeur peut décider du nombre de tests à sélectionner en fonction du niveau de confiance requis et des ressources disponibles.
- En supposant que tous les tests ont la même efficacité en termes de détection des erreurs, la confiance dans l'exactitude du code inchangé augmente avec le nombre de tests échantillonnés et exécutés.
  - Cette hypothèse a peu de chances de se vérifier en pratique. En effet, certains tests échantillonnés peuvent n'avoir aucun lien avec le code modifié, tandis que d'autres pourraient en avoir un.

Sélection des tests de traversée de modification

- Lorsqu'une petite modification est apportée au logiciel, quelles parties du logiciel peuvent être impactées par cette modification ?
- Évaluez la propagation des modifications dans le logiciel.

Analyse d'impact des changements

Quel est l'impact d'un changement sur le reste du logiciel ?

- Sélectionnez un sous-ensemble de T de telle sorte que seuls les tests qui garantissent l'exécution du code modifié et du code qui pourrait être impacté par le code modifié dans P soient sélectionnés tandis que ceux qui ne le font pas sont rejetés.

Sélection des tests de traversée de modification

L'objectif est d'identifier les cas de test qui traversent les parties **modifiées**.

- **Phase 1** : P est exécutée et la trace est enregistrée pour chaque cas de test dans Tno = Tu ∪ Tr.

- **Phase 2 :** Tr est isolé de Tno par une comparaison de P et P' et une analyse des traces d'exécution
  - **Étape 2.1 :** Construire des arbres de CFG et de syntaxe
  - **Étape 2.2 :** Comparer les CFG et sélectionner des tests

Obtain Execution Traces

```
main () {
 int x, y, p;
 input (x, y);
 if (x < y)
    p = g1(x, y);
 else
    p = g2(x, y);
 endif
 output (p);
end
}
```

```
int g1 (int a, b) {
int a, b;
if (a + 1 == b)
    return (a*a);
else
    return (b*b);
```

```
int g2 (int a, b) {
int a, b;
if (a == (b + 1))
    return (b*b);
else
    return (a*a);
```

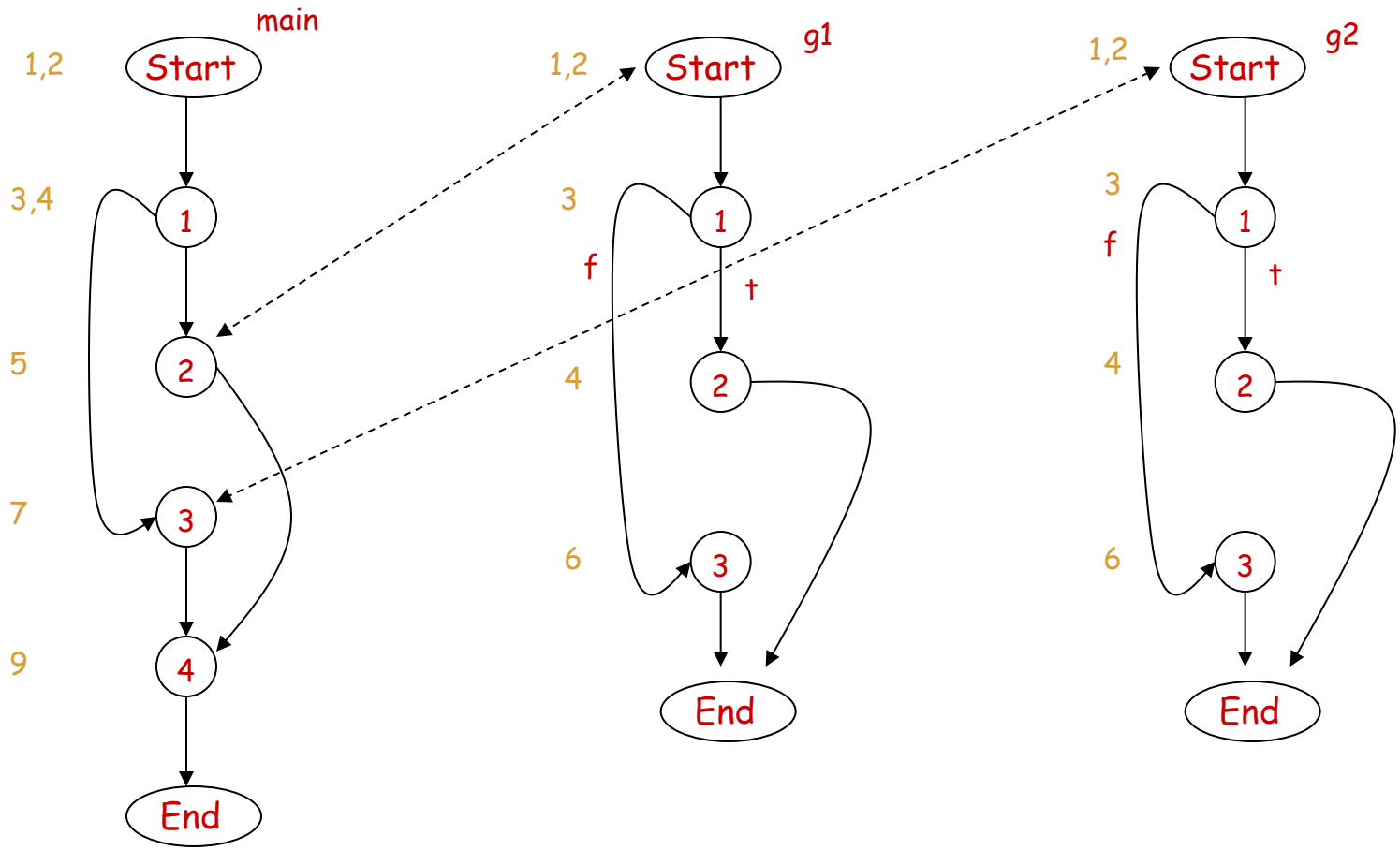
Consider the following test set:

t1: <x=1, y=3>

t2: <x=2, y=1>

t3: <x=1, y=2>

CFG


main
1,2
Start
3,4
1
5
2
7
3
9
4
End
g1
1,2
Start
3
1
f
t
4
2
6
3
End
g2
1,2
Start
3
1
f
t
4
2
6
3
End

Execution Trace

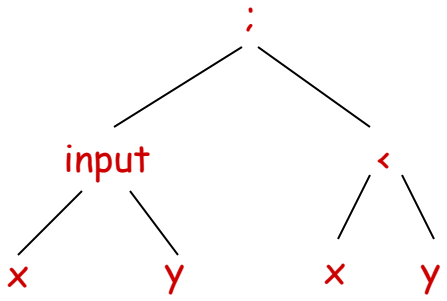
Test (t)	Execution Trace (trace(t))
t1	main.Start, main.1, main.2, g1.Start, g1.1, g1.3, g1.End, main.2, main.4, main.End
t2	main.Start, main.1, main.3, g2.Start, g2.1, g2.2, g2.End, main.3, main.4, main.End
t3	main.Start, main.1, main.2, g1.Start, g1.1, g1.2, g1.End, main.2, main.4, main.End

Test Vector

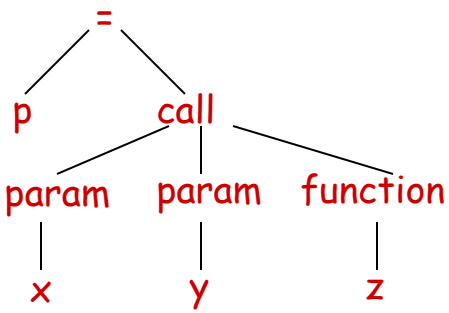
	Test vector (test(n)) for node n			
Function	1	2	3	4
main	t1, t2, t3	t1, t3	t2	t1, t2, t3
g1	t1, t3	t3	t1	-
g2	t2	t2	None	-

Les colonnes sont les **numéros de nœud** dans chaque fonction.

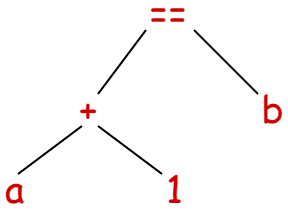
Syntax Tree



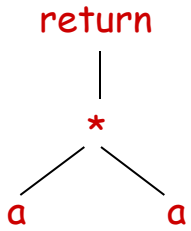
main.1



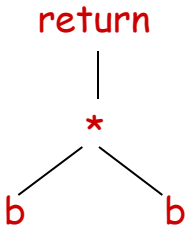
main.2



g1.1



g1.2 and g2.3



g1.3 and g2.2

- Un point-virgule indique la séquence de deux lignes ou plus dans un nœud.
- Dans un arbre de syntaxe, un appel de fonction est représenté par des nœuds de paramètres, un pour chaque paramètre, et un nœud d'appel.

Stratégie de sélection

- Les CFG pour **P** et **P'** sont comparés aux nœuds d'identification qui diffèrent en **P** et **P'**.
  - Deux nœuds sont considérés comme **équivalents** si les arbres de syntaxe correspondants sont **identiques**.
  - Deux arbres de syntaxe sont considérés comme **identiques** lorsque leurs racines portent les mêmes étiquettes et les mêmes descendants correspondants.
- Les tests qui traversent ces nœuds sont sélectionnés.

Example

Considérons l'exemple précédent. Supposons que la fonction **g1** soit modifiée comme suit :

```
1. int g1 (int a, b) {
2. int a, b;
3. if (a - 1 == b) ← Predicate modified
4.     return (a*a);
5. else
6.     return (b*b);
```

Quels sont les tests sélectionnés ?

Example

```
1. int g1 (int a, b) {
2. int a, b;
3. if (a - 1 == b) ← Predicate modified
4.     return (a*a);
5. else
6.     return (b*b);
```

	Test vector (test(n)) for node n			
Function	1	2	3	4
main	t1, t2, t3	t1, t3	t2	t1, t2, t3
g1	t1, t3	t3	t1	-
g2	t2	t2	None	-

Quels tests traversent ce nœud modifié ?

Pour g1, le **nœud 1** (celui qui correspond au test du if dans cet exemple) est traversé par : **t1, t3**

Minimisation des tests

- **Élimine** les tests apparemment redondants par rapport à certains critères (par exemple, le flux de contrôle, la couverture du flux de données ou la couverture des exigences).
- **Exemple** : si t1 et t2 testent tous les deux la fonction f dans P, on peut alors décider d'abandonner t2 au profit de t1.
- L'objectif de la minimisation est de réduire le nombre de tests de régression à exécuter.

Étant donné un ensemble de test **T**, est-il possible de réduire **T** à **T'** de sorte que **T'** $\subseteq$ **T** et **T'** couvrent toujours toutes les entités testables couvertes par **T** ?

Minimisation des tests

- Les tests à conserver peuvent être basés sur plusieurs politiques :
  - Tests associés aux rapports de problèmes clients
  - Garantir qu'un critère de couverture est toujours respecté
- Parfois plus difficile de choisir les tests à supprimer
  - Supprimez les tests qui ne contribuent pas à la couverture.
  - Supprimez les tests qui n'ont jamais détecté de défaut (risqué !).
  - Supprimez les tests qui ont détecté le même défaut que d'autres tests (risqué aussi !).
- La minimisation des tests n'est pas forcément sûre.
- Il est recommandé d'effectuer la minimisation avec prudence.
  - Les tests rejetés par un algorithme de minimisation doivent être examinés attentivement avant d'être définitivement rejetés.

Exemple de minimisation des tests

- Exemple : P' est nécessaire pour générer la **somme** de deux entiers d'entrée. Suite à une erreur, le programme génère la **différence** des deux entiers.

1. int x, y;
2. entrée x, y;
3. sortie (x-y);

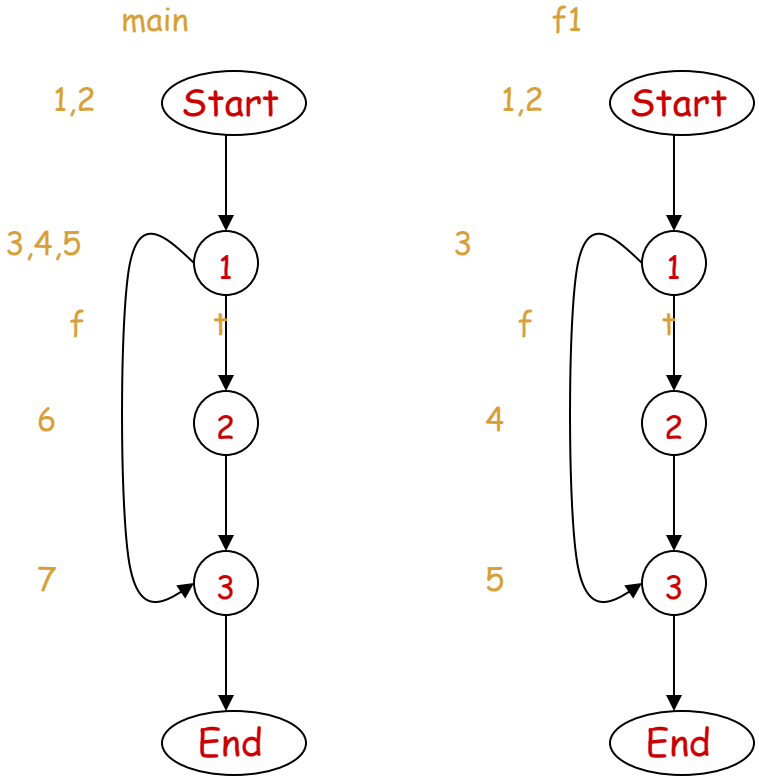
- Supposons que Tr contienne 10 tests, dont 9 avec y = 0 et 1 TC (tnz) avec des valeurs non nulles pour x et y. tnz est le seul test qui fait échouer P'.
- **Supposons que Tr soit minimisé de sorte que la couverture de bloc de base obtenue en exécutant P' reste inchangée.**
- Chacun des 10 tests de Tr **couvre le seul bloc de base de** P' ; par conséquent, tous les tests sauf un seront éliminés par un algorithme de minimisation. Si tnz est éliminé, l'erreur de P' ne sera pas révélée par la suite de tests minimisée.

Example (1)

```
main () {
  int x, y, z;
  input (x, y);
  z = f1(x);
  if (z > 0)
     z = f2(x);
  output (z);
end
}
```

```
int f1(int x) {
  int p;
  if (x > 0)
     p = f3(x, y);
  return (p);
}
```

Example (2)



Consider the following test set:
t1: main: 1, 2, 3; f1 : 1, 3
t2: main: 1, 3; f1: 1, 3
t3: main: 1, 3; f1: 1, 2, 3

Example (3)

Consider the following test set:
t1: main: 1, 2, 3; f1 : 1, 3
t2: main: 1, 3; f1: 1, 3
t3: main: 1, 3; f1: 1, 2, 3

Données (couverture)

- **t1** : main {1,2,3} ; f1 {1,3}
- **t2** : main {1,3} ; f1 {1,3}
- **t3** : main {1,3} ; f1 {1,2,3}

1) Couverture totale de T

main : $\{1, 2, 3\}$; **f1** :$\{1, 2, 3\}$

Donc l'objectif est un $T' \subseteq T$ qui couvre toujours **main {1,2,3}** et **f1 {1,2,3}**.

2) Trouver les éléments "uniques" (obligatoires)

main:2 n'est couvert **que par t1 ⇒ t1 est obligatoire**

f1:2 n'est couvert **que par t3 ⇒ t3 est obligatoire**

Donc T' doit contenir **t1 et t3**.

Priorisation des tests

- Il s'agit de hiérarchiser les cas de test selon certains critères.
- Une métrique appropriée est utilisée pour classer tous les tests dans Tr.
- Le test le plus élevé a la priorité la plus élevée, celui qui vient après a la deuxième priorité, et ainsi de suite.
- La priorisation n'élimine aucun test de Tr. Elle permet au testeur de choisir les tests à sélectionner en fonction de leur priorité relative et de leurs relations individuelles, en fonction du séquençage et d'autres exigences.

Priorisation des tests (suite)

- Exemple : soit R1, R2 et R3, trois exigences reportées sans changement de P vers P'.
- Une approche classe d'abord les exigences de P' selon leur criticité :

  R2 est la plus critique, suivie de R3, et enfin de R1.
- **Les nouvelles exigences implémentées dans P' mais non dans P ne sont pas utilisées dans ce classement, car elles correspondent à de nouveaux tests Td.**
- Supposons que le sous-ensemble de régression **Tr** de P soit {T1, T2, T3, T4, T5}.
  - T1 teste R1,
  - T2 et T3 testent R2,
  - T4 et T5 testent R3.

Priorisation des tests (suite)

- Nous pouvons maintenant prioriser les tests dans cet ordre :

T2, T3, T4, T5, T1

où **T2** est la **priorité** la plus élevée et **T1** la **plus basse.**

- Si le testeur ne dispose que des ressources nécessaires pour exécuter trois tests avant la publication de P', alors T2, T3 et T4 peuvent être sélectionnés.

- De nombreuses techniques ont été proposées pour prioriser les tests, dont certaines utilisent **la quantité de code couvert** comme indicateur.

Automatisation et support des outils

- Trop de tests à exécuter manuellement
- Les tests doivent être exécutés et évalués rapidement,
  - souvent pendant la nuit, ou plus fréquemment pour les applications web.
- Les testeurs n'ont pas le temps d'examiner les résultats par inspection.

Les tests de régression doivent être automatisés

Automatisation et support des outils

- Types d'outils :
  - Spécification des cas de test et des résultats, par exemple, Junit
  - Capture/Replay – Capture des valeurs saisies dans une interface graphique et relecture de ces valeurs sur les nouvelles versions.
  - Contrôle de version – Suivi des collections de tests, des résultats attendus, de leur origine, des critères utilisés et de leur efficacité passée.
  - Logiciel de script – Gestion du processus d'obtention des données d'entrée, d'exécution du logiciel, d'obtention des données de sortie, de comparaison des résultats et de génération des rapports de test.

Résumé de trois outils pour aider aux tests de régression

Table 9.1 A summary of three tools to aid in regression testing.

Attributes	Atac/χ Suds	TestTube	Echelon
Developer	Telcordia Technologies	AT&T Bell Laboratories	Microsoft
Year reported	1992	1994	2002
Selective retesting	Yes	Yes	No
Selection basis	Control/data flow coverage	Functions	Basic blocks
Test prioritization	Yes	No	Yes
Test minimization	Yes	No	No
Slicing	Yes	No	No
Differencing	Yes	Yes	Yes
Block coverage	Yes	No	Yes
Condition coverage	Yes	No	No
def-use coverage	Yes	No	No
p-use, c-use coverage	Yes	No	No
Language support	C	C	C and binary

Mathur, Aditya P. *Foundations of software testing, 2/e*. Pearson Education India, 2013.

Autres outils pour faciliter les tests de régression

- Outil : **Selenium**

Lien : http://seleniumhq.org/

Tests de régression et de système d'applications Web.

- Outil : **SilkTest**

Lien : http://www.borland.com/us/products/silk/index.aspx

SilkTest permet d'automatiser les tests pour les applications Web et autres.

Lorsqu'un test de régression échoue

- Les modifications apportées aux interfaces externes peuvent parfois entraîner l'échec de tous les tests.
  - Les outils modernes de capture/relecture ne se laissent pas tromper par des modifications triviales comme la couleur, le format et le placement.
  - Les scripts automatisés peuvent être modifiés automatiquement via des modifications globales dans un éditeur ou par un autre script.
- Les tests de régression sont évalués selon que le résultat du nouveau programme P est équivalent ou non à celui de la version précédente P-1.
  - En cas de différence, le test est considéré comme ayant échoué.

Lorsqu'un test de régression échoue

- Les échecs des tests de régression peuvent être dus à trois raisons :
  - Le logiciel présente un défaut – Il faut le corriger ;
  - Les valeurs du test ne sont plus valides dans la nouvelle version – Il faut supprimer ou modifier le test ;
  - Le résultat attendu n'est plus valide – Il faut mettre à jour le test.

- Parfois difficile de décider lequel !!

Exercice

- Considérons le test de régression, supposons que réexécuter un test prenne 10 minutes. Supposons que 70 % des tests devront être réexécutés à chaque cycle de tests de régression.
- Considérez une technique de sélection de tests de régression donnée pour décider quels tests effectuer ou non. Supposons que nous ayons **100 cas de test** et qu'il faut **m** minutes pour décider si un cas de test doit être réexécuté ou non.

Quelle est la borne supérieure du temps de décision de la technique par test (c'est-à-dire la valeur de m) pour qu'elle soit considérée comme efficace ? Justifiez.

Exercice: solution

- Supposons 100 cas de test. À chaque tour de test de régression, seulement 70 seraient pertinents et devraient être pris en compte. 30 tests devraient être écartés. Supposons qu'il faut m minutes pour décider si le cas de test doit être relancé ou non.
- Temps total d'utilisation de la technique:
  - $T = 100 * m + 70 * 10 = 100\, m + 700$
- Temps total si on n'utilise pas la technique de sélection:
  - $O = 100 * 10 = 1000$
- Pour que la technique soit efficace:

$T < O \rightarrow 100\, m + 700 < 1000 \rightarrow m < 3$ minutes